

1 Uvod

V tem diplomskem delu, bomo raziskali, kaj je linearna logika in kako se jo uporablja v računalništvu. Linearna logiko je prvi predstavil Jean-Yves Girard [2]. Zelo hitro se je izkazala za uporabno, saj zahteva, da smo pozorni na kaj imamo na voljo v danem trenutku, kar bolj odseva realni svet. Zato linearni logiki pravimo tudi *logika stanja*.

Preko Curry-Howardovega izomorfizma vemo, da lahko logične dokaze interpretiramo kot računalniške programe.

2 Naravna dedukcija

V tem poglavju bomo najprej predstavili pravila sklepanja na katerih temelji naravna dedukcija oziroma “standardna” logika in s tem bomo tudi predstavili notacijo, ki bo uporabljena v tem diplomskem delu.

Konjunkcija. Pravilo vpeljave za konjunkcijo pravi, če sta izjavi A in B res, potem je tudi izjava A in B res. To lahko zapišemo kot

$$\frac{A \text{ res} \quad B \text{ res}}{A \wedge B \text{ res}} \wedge I.$$

Nad črto zapišemo končno število predpostavk in pod črto zapišemo sklep. Desno od črte zapišemo ime pravila, ki ga ponavadi okrajšamo. V našem primeru $\wedge I$ predstavlja pravilo vpeljave za konjunkcijo (ang. conjunction introduction). Splošna oblika pravila sklepanja je torej oblike

$$\frac{\mathcal{H}_1 \quad \mathcal{H}_2 \quad \dots \quad \mathcal{H}_n}{\mathcal{S}} \text{ ime}$$

kjer so $\mathcal{H}_1, \dots, \mathcal{H}_n$ *premise* in $\mathcal{S}$ sklep. Izjavam $\mathcal{S}, \mathcal{H}_1, \dots, \mathcal{H}_n$ pravimo sodbe [3, Section 2]. Kasneje bomo videli kako lahko taka pravila sestavljamo skupaj, da dobimo izpeljana pravila. Takim zapisov dokazov pravimo *sklepna drevesa*. Navadno pišemo zraven še kontekst, ki ga ponavadi označimo z Γ . Sodba $\Gamma \vdash A$ pravi, da iz konteksta Γ lahko vzamemo izjavo A in jo uporabimo.

$$\frac{\Gamma \vdash A \text{ res} \quad \Gamma \vdash B \text{ res}}{\Gamma \vdash A \wedge B \text{ res}} \wedge I.$$

Torej to preberemo, kot če imamo poljubni resnični izjavi A in B , potem v tem kontekstu velja tudi $A \wedge B$. V tem poglavju zaradi preglednosti ne bomo pisali zraven konteksta, vendar bo v kasnejših poglavjih igral pomembno vlogo za razumevanje.

Za konjunkcijo imamo dva pravila uporabe (ang. elimination rule).

$$\frac{A \wedge B \text{ res}}{A \text{ res}} \wedge E_L \quad \frac{A \wedge B \text{ res}}{B \text{ res}} \wedge E_D$$

Implikacija. Malce bolj zahtevno pravilo vpeljave ima implikacija. Pravilo se glasi, če lahko iz A izpeljemo B potem velja $A \Rightarrow B$. To zapišemo kot

$$\frac{\begin{array}{c} \cdot \\ \vdots \\ B \text{ res} \end{array}}{A \Rightarrow B \text{ res}} \Rightarrow I^u$$

Tri pike $:$ predstavljajo, da iz A pokažemo B . Na vrhu $\cdot$ predstavlja prazen nabor premis. Navadno se pike ne piše in se pusti prostor nad črto prazen. Tu je napisana, da poudarimo, da nimamo začetnih pogojev. Ne predpostavimo, da A velja in potem pokažemo B , temveč brez dodatnih predpostavk pokažemo, da iz A sledi B . Končno pravilo označimo z $\Rightarrow I^u$, da poudarimo, da implikacija pride iz vrhnjega pravila označenega s črko u . Pomembno je, da A , kasneje v dokazu ne moremo več uporabiti, namreč predpostavili smo A samo zato da lahko dokažemo $A \Rightarrow B$, vendar A ne drži nujno na splošno. Zato takoj ko uporabimo pravilo I^u , se veja drevesa od u do I^u zapre. Nikjer drugje ne moremo uporabiti več sodb, ki smo jih dobili pod pogojem da smo predpostavili A .

Za lažje razumevanje notacije pravila si pogledjmo primer dokaza za $A \Rightarrow (B \Rightarrow (A \wedge B))$ [3, stran 6].

$$\frac{\frac{\frac{\overline{A \text{ res}}^u \quad \overline{B \text{ res}}^w}{A \wedge B \text{ res}} \wedge I}{B \Rightarrow (A \wedge B) \text{ res}} \Rightarrow I^w}{A \Rightarrow (B \Rightarrow (A \wedge B))} \Rightarrow I^u$$

Pri tem primeru bi radi poudarili strukturo *drevesa dokaza*. Pravila dokazovanja se gradijo v večja *izpeljana pravila* v obliki usmerjenega drevesnega grafa, kjer so vozlišča sodbe in povezave pravila med njimi. Razvidno je tudi, da trditev $A \Rightarrow (B \Rightarrow (A \wedge B))$ drži sama po sebi in nismo potrebovali nobenih premis, da smo to pokazali, pravili u in v pa sta le simbolne narave in delujeta kot “predpostavimo da, ...”.

Pravilo uporabe za implikacijo je enostavnejše.

$$\frac{A \Rightarrow B \text{ res} \quad A \text{ res}}{B \text{ res}} \Rightarrow E$$

Disjunkcija. Disjunkcija ima dva pravila vpeljave.

$$\frac{A \text{ res}}{A \vee B \text{ res}} \vee I_L \quad \frac{B \text{ res}}{A \vee B \text{ res}} \vee I_D$$

Ko v dokazu želimo uporabiti disjunkcijo, ločimo na dva primera, če velja desna ali leva stran disjunkcije.

$$\frac{\frac{\overline{A \text{ res}}^u \quad \overline{B \text{ res}}^w}{\vdots} \quad \frac{\overline{C \text{ res}}^w}{\vdots}}{\frac{A \vee B \quad C \text{ res}}{C \text{ res}}} \vee E^{u,w}$$

Vidimo da je to pravilo sklepanja podobno pravilu uporabe za implikacijo, saj je spet potrebno pokazazati, da iz A lahko dobimo C , kot da bi vpeljali implikacijo. In spet oznaka $E^{u,v}$ pove, da ne moremo uporabiti sodb na vejah med u in $E^{u,v}$ in med w in $E^{u,v}$ nikjer drugje. Pogledjmo si primer kako uporabimo implikacijo, tako da dokažemo $(A \vee B) \Rightarrow (B \vee A)$.

$$\frac{\frac{\overline{A \vee B \text{ res}}^u}{\vdots} \quad \frac{B \vee A \text{ res}}{(A \vee B) \Rightarrow (B \vee A)}}{\Rightarrow I^u}$$

Nadaljujemo na edini način, ki nam je na voljo in sicer razdelitev $A \vee B$ na primere.

$$\begin{array}{c}
\frac{}{A \text{ res}}^w \quad \frac{}{B \text{ res}}^v \\
\vdots \quad \vdots \\
\frac{\frac{}{A \vee B \text{ res}}^u \quad \frac{}{B \vee A \text{ res}} \quad \frac{}{B \vee A \text{ res}}}{B \vee A \text{ res}} \vee E^{v,w} \\
\frac{}{(A \vee B) \implies (B \vee A)} \Rightarrow I^u
\end{array}$$

Od tu je nadaljevanje preprosto, saj uporabimo le pravila vpeljave za disjunkcijo.

$$\begin{array}{c}
\frac{}{A \vee B \text{ res}}^u \quad \frac{\frac{}{A \text{ res}}^w}{B \vee A \text{ res}} \vee I_D \quad \frac{\frac{}{B \text{ res}}^v}{B \vee A \text{ res}} \vee I_L \\
\frac{}{B \vee A \text{ res}} \vee E^{v,w} \\
\frac{}{(A \vee B) \implies (B \vee A)} \Rightarrow I^u
\end{array}$$

V kasnejših poglavjih bomo povedali še pravila za eksistenčni in univerzalni kvantifikator ter za negacijo.

3 Curry-Howardov izomorfizem

Matematične dokaze iz logike lahko interpretiramo kot računalniške programe ali pa kot sklepe v teoriji tipov. Notacija je zelo podobna, kot ta ki smo jo obravnavali v prejšnjem poglavju. Spodaj je primerjava pravil sklepanja.

Produktni tip.

$$\begin{array}{c}
\frac{\Gamma \vdash A \text{ res} \quad \Gamma \vdash B \text{ res}}{\Gamma \vdash A \wedge B \text{ res}} \wedge I \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash (a, b) : A \times B} \times I \\
\\
\frac{\Gamma \vdash A \wedge B \text{ res}}{\Gamma \vdash A \text{ res}} \wedge E_L \quad \frac{\Gamma \vdash x : A \times B}{\Gamma \vdash fst(x) : A} \times E_1 \\
\\
\frac{\Gamma \vdash A \wedge B \text{ res}}{\Gamma \vdash A \text{ res}} \wedge E_D \quad \frac{\Gamma \vdash x : A \times B}{\Gamma \vdash snd(x) : B} \times E_2
\end{array}$$

Na tej točki je spet potrebno pojasniti nekaj notacije. Kot že rečeno, Γ predstavlja kontekst, torej končen nabor predpostavk. $\Gamma \vdash A$ se imenuje *sekvent* [4, L2.8] in pravi, da iz konteksta Γ lahko vzamemo A , da ga uporabimo naprej. Podobno interpretiramo sekvent tudi v teoriji tipov. Se pravi $\Gamma \vdash a : A$ v kontekstu G lahko uporabimo element a tipa A , kjer je kontekst v tem primeru končen nabor tipov, katerih elemente lahko uporabljamo.

Izpostavimo dejstvo, da nismo povedali “kaj” je produktni tip, le kako ga konstruiramo in uporabljamo, namreč ni zares pomembno kako izgleda posamezen tip, vendar kako ga uporabljamo. Torej na nek način so tipi bolj abstraktna stvar kot izjave. Ampak kljub temu, nam povedo nekaj več, namreč kljub temu, da imamo večji poudarkih na tipih, operiramo z elementi. Pravila sklepanja nam v teoriji tipov pokažejo kako konstruiramo konkretne elemente iz elementov drugih tipov. Torej iz a tipa A in iz b tipa B vemo kako konstruirati element (a, b) tipa $A \times B$. To lahko naredimo, ker v teoriji tipov nimamo le pravil vpeljave in uporabe, temveč tudi pravila računanja. S temi pravili dobimo predpis, kako dobimo $a : A$ in $b : B$ iz $A \times B$.

$$fst((a, b)) \equiv a$$

$$snd((a, b)) \equiv b$$

Funkcijski tip.

$$\begin{array}{c}
\frac{}{\Gamma \vdash A \text{ res}}^u \\
\vdots \\
\frac{\Gamma \vdash B \text{ res}}{\Gamma \vdash A \Rightarrow B \text{ res.}} \Rightarrow I^u \\
\\
\frac{\Gamma \vdash A \Rightarrow B \text{ res} \quad \Gamma \vdash A \text{ res}}{\Gamma \vdash B \text{ res}} \Rightarrow E \\
\\
\frac{}{\Gamma \vdash x : A}^u \\
\vdots \\
\frac{\Gamma \vdash b : B}{\Gamma \vdash \lambda x : A. b : A \rightarrow B} \rightarrow I^u \\
\\
\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B} \rightarrow E
\end{array}$$

Pri vpeljavi funkcijskega tipa, je pomembna spremenljivka $x : A$. Pokazati moramo, da iz katerega koli elementa $x : A$ lahko dobimo nek element $b : B$, ki je lahko odsviden od $x : A$. Potem lahko skonstruiramo funkcijo, ki vsakemu x -u predpiše pripadajoč b . Predpis $\lambda x : A. b : A \rightarrow B$ je v λ -notaciji. Pove da vezemo spremenljivko x tipa A in vrnemo izraz b . To je funkcija tipa $A \rightarrow B$.

Pri pravilu uporabe za funkcijo je sklep zapisan z $f a : B$. To je ekvivalentno temu, da bi pisali $f(a)$, vendar v teoriji tipov se navadno piše brez oklepajev.

Pravilo računanja za funkcijski tip nam pove, kako se obnaša λ -račun.

$$(\lambda x : A. b)u \equiv b[x/u]$$

Operacijo $b[x/u]$ imenujemo β -redukcija, pomeni pa, da vse pojavitve x v b zamenjamo z u , torej kot da bi evalvirali funkcijo $b(x)$ v točki u . Sicer b ni nujno funkcijskega tipa, vendar je vseeno lahko odvisna od x . Pozorni moramo biti, da ponesreči ne spremenimo pomena elementa $b : B$, namreč $(\lambda x : A. \lambda y : B. x)[y/x] \neq \lambda x : A. \lambda x : B. x$ ¹. Najprej moramo vse vezane spremenljivke preimenovati.

$$(\lambda x : A. \lambda y : B. x)[y/x] = (\lambda z : A. \lambda y : B. z)[y/x] = \lambda z : A. \lambda x : B. z$$

Pri tem primeru bodio še pozorni na to, da tu ne evalviramo funkcije v neki točki, temveč preimenujemo vezane spremenljivke. Če bi želeli evalviratu ta lambda račun v nekem elementu $y : A$ bi napisali

$$(\lambda x : A. \lambda y : B. x)y = (\lambda y : B. x)[x/y] = (\lambda z : B. x)[x/y] = (\lambda z : B. y).$$

Dobili bi torej λ -funkcijo, ki vse elemente iz B slika v $y : A$.

Vsotni tip.

$$\begin{array}{c}
\frac{\Gamma \vdash A \text{ res}}{\Gamma \vdash A \vee B \text{ res}} \vee I_L \quad \frac{\Gamma \vdash a : A}{\Gamma \vdash in_1(a) : A + B} +I_L \\
\\
\frac{\Gamma \vdash B \text{ res}}{\Gamma \vdash A \vee B \text{ res}} \vee I_D \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash in_2(b) : A + B} +I_D
\end{array}$$

¹Izraz na desni strani neenakosti ni validen, saj dvakrat vezemo isto spremenljivko

$$\begin{array}{c}
\frac{}{\Gamma \vdash A \text{ res}}^u \quad \frac{}{\Gamma \vdash B \text{ res}}^w \\
\vdots \quad \vdots \\
\frac{\Gamma \vdash A \vee B \quad \Gamma \vdash C \text{ res} \quad \Gamma \vdash C \text{ res}}{\Gamma \vdash C \text{ res}} \vee E^{u,w} \\
\\
\frac{}{\Gamma \vdash x : A}^u \quad \frac{}{\Gamma \vdash y : B}^w \\
\vdots \quad \vdots \\
\frac{\Gamma \vdash c : A + B \quad \Gamma \vdash a : C \quad \Gamma \vdash b : C}{\Gamma \vdash (\text{case } c \text{ of } in_1(x) \mapsto a | in_2(y) \mapsto b) : C} +E^{u,w}
\end{array}$$

Tako kot pri funkcijskem tipu, moramo najti način, da za vsak element $x : A$ lahko dobimo $a : C$ in za vsak element $y : B$ najdemo $b : C$. To nam zagotovi, da lahko za element $c : A + B$ najdemo nek element tipa C .

Za vsotni tip imamo dva računska pravila.

$$\begin{array}{l}
\text{case } in_1(s) \text{ of } in_1(x) \mapsto t | in_2(y) \mapsto r \equiv t[x/s] \\
\text{case } in_2(s) \text{ of } in_1(x) \mapsto t | in_2(y) \mapsto r \equiv r[y/s]
\end{array}$$

4 Linearna naravna dedukcija

Sedaj lahko predstavimo linearno logiko. Glavna lastnost v linearni logiki je, da vsako premiso uporabiš natanko enkrat. Na prvi pogled se to zdi nenavadno, saj si resnico predstavljamo, kot neodvisno od tega ali smo jo že uporabili ali ne. Vendar videli bomo, da linearna logika v veliko primerih bolje opiše svet.

Recimo, da stojimo pred avtomatom. Avtomat sprejema samo kovance za 50c in 1€ in potrebno je vstaviti točen znesek. Na avtomatu si lahko kupimo čokoladico za 1€, bombončke za 50c ali pa sladko pijačo za 1€. To lahko predstavimo s pravili sklepanja.

$$\begin{array}{ccc}
\frac{1\text{€}}{\text{pijača}} & \frac{1\text{€}}{\text{čokoladica}} & \frac{50\text{c}}{\text{bombončki}}
\end{array}$$

Recimo, da imamo tri kovance za 1€ in štiri kovance za 50c. Skupina naslednjih sklepov je veljavna v linearni logiki.

$$\begin{array}{ccccc}
\frac{1\text{€} \quad 50\text{c}}{\text{pijača}} & \frac{1\text{€}}{\text{čokoladica}} & \frac{50\text{c} \quad 50\text{c}}{\text{čokoladica}} & \frac{50\text{c}}{\text{bombončki}} & \frac{1,5\text{€}}{1,5\text{€}}
\end{array}$$

Vidimo, da se vrednost ne izgubi. Začeli smo z 5€ in dobili ven 5€ vrednosti stvari. Ne moremo ničesar izgubiti in prav tako ne moremo ničesar pridobiti. Istega kovanca ne moremo uporabiti dvakrat, torej so sklepi, ki jih imamo na voljo odvisni od stanja virov in vsakič ko uporabimo sklep spremenimo stanje virov.

Hkratna konjunkcija. Recimo da lahko s pomočjo konteksta Γ pokažemo izjavo A in s pomočjo konteksta Δ pokažemo izjavo B , Potem lahko s kontekstom Γ, Δ pokažemo da velja A in B . Oznaka za tako konjunkcijo v linearni logiki je tenzor $A \otimes B$.

$$\frac{\Gamma \vdash A \quad \Delta \vdash B}{\Gamma, \Delta \vdash A \otimes B} \otimes I$$

Konteksta Γ in Δ združimo, oziroma *konkateriramo* glede na spodnja pravila.

$$\Gamma, (\Delta, u : A) = (\Gamma, \Delta), u : A = (\Gamma, u : A), \Delta$$

Pri zgornji enakosti “,” predstavlja dve različni operaciji. Ena operacija združi kontekste, druga pa kontekstu pridruži izjavo imenovano u . V našem primeru kovancev, bi A predstavljal “imam kovanec za 1€”, specifični konvenac, ki dokazuje to trditev, pa je označen z u .

Glede na to, da $\otimes$ predstavlja konjunkcijo, bi lahko zmotno mislili, da je pravilo uporabe za $\otimes$, enako kot za $\wedge$, vendar pravilo za uporabo *land* izgubi del informacije.

$$\frac{A \wedge B \text{ res}}{A \text{ res}} \wedge E_L$$

Na vrhu imamo A in tudi B , vendar v sklepu uporabimo samo A . Ne seldimo pravilom linearne logike, ki pravi, da je potrebno vsako premiso uporabiti natanko enkrat. Pravilo vpeljave hkratne konjunkcije pravi, če lahko dosežemo cilj C s pomočjo virov $u : A$ in $w : B$ in če imamo $A \otimes B$, lahko dosežemo cilj C .

$$\frac{\Gamma \vdash A \otimes B \quad \Delta, u : A, w : B \vdash C}{\Gamma, \Delta \vdash C} \otimes E^{u,w}$$

Oznaka $U^{u,w}$ predstavlja, da sta vira označena z u in w porabljena, torej ju kasneje ne moremo več uporabiti. Na tej točki je smiselno omeniti, kaj v linearni logiki pomeni kontekst. To je tenzorski produkt *virov* $A_1 \otimes \dots \otimes A_n = \Gamma$. Navadno tenzorjev ne pišemo in vire preprosto ločimo z vejico $A_1, \dots, A_n$. Torej to nam pove, da morajo viri v kontekstu biti resnični oziroma na voljo v istem stanju.

Alternativna konjunkcija. Recimo da imamo 1€. Potem sli lahko kupimo ali čokoladico ali pijačo, vendar ne oboje na enkrat.

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B} \&I$$

To pravilo vpeljave izgleda enako kot pravilo vpeljave za konjunkcijo $\wedge$, vendar pomen je drugačen, saj $A \& B$ predstavlja “natanko eno od A in B ”.

Če imamo vire za $A \& B$, vemo da imamo vire za natanko eno od A in B . Torej pravili uporabe izgledata takole:

$$\frac{\Gamma \vdash A \& B}{\Gamma \vdash A} \&E_L \qquad \frac{\Gamma \vdash A \& B}{\Gamma \vdash B} \&E_D$$

Linearna implikacija. [1, poglavje 1.2] Za linearno implikacijo se uporablja znak $\multimap$ imenovan *lizika* [4, L20.3].

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \multimap B} \multimap I.$$

Linearna implikacija $A \multimap B$ porabi vir A , vendar implikacijo sama ostane, torej če imamo na voljo še kakšen vir A , ga lahko ponovno uporabimo.

$$\frac{\Gamma \vdash A \quad \Delta \vdash A \multimap B}{\Gamma, \Delta \vdash B}$$

Pravilo uporabe za linearno implikacijo upošteva, da se viri, ki jih uporabimo, da dobimo A in viri, da dobimo $A \multimap B$ lahko prekrivajo.

Sedaj lahko pokažemo eno pomembno pravilo imenovano *rez*. Intuitivno se nam zdi, da če z viri Γ, A lahko dosežemo B in z viri Γ lahko dosežemo A , potem bi viri Γ zadostovali, da dosežemo B .

$$\frac{\Gamma \vdash A \quad \Gamma, A \vdash B}{\Gamma \vdash B} \text{ cut}$$

In res s pravili iz poglavja Naravna dedukcija lahko izpeljemo rez.

$$\frac{\Gamma \vdash A \quad \frac{\Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B} \Rightarrow I}{\Gamma \vdash B} \Rightarrow E$$

V linearni logiki je to pravilo rahlo drugačno, saj pri $\Rightarrow I$ uporabimo dvakrat isti kontekst, medtem ko pri $\multimap I$ tega ne smemo narediti.

$$\frac{\Gamma \vdash A \quad \frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B} \multimap I}{\Gamma, \Delta \vdash B}$$

Izpeljano pravilo je torej

$$\frac{\Gamma \vdash A \quad \Delta, A \vdash B}{\Gamma, \Delta \vdash B} \text{ cut.}$$

Disjunkcija (zunanja izbira).

$$\frac{\Gamma \vdash B}{\Gamma, A \vdash B} \text{ weak}$$

5 Slovarček

judgment sodba

premise premisa

conclusion sklep

inference rule pravilo sklepanja

computation rule pravilo računanja

β -reduction β -redukcija

resource vir

simultaneous conjunction hkratna konjunkcija

alternative conjunction alternativna konjunkcija

cut rez

lolli lizika

external choice zunanja izbira

Literatura

- [1] Andrew G Barber. Linear type theories, semantics and action calculi. 1997.
- [2] Jean-Yves Girard. Linear logic. *Theoretical computer science*, 50(1):1–101, 1987.
- [3] Frank Pfenning. Lecture notes on natural deduction. *Course CMU*, 15317, 2009.
- [4] Frank Pfenning. Lecture notes on truth is ephemeral. 2023.